

APPLICATION NOTE

Linux 系统 ATBM iwpriv 命令使用说明



**ATBM6062
C**

1x1 802.11b/g/n/
ax
Wi-Fi 芯片

Table of contents

目录

1	开始发包命令.....	6
2	停止发包命令.....	6
3	开始收包命令.....	6
4	停止收包命令.....	7
5	开始发送 SINGLTONE 命令.....	7
6	停止 SINGLTONE 命令.....	7
7	配置发送功率.....	7
8	设置信号发射功率 1.....	8
9	设置信号发射功率 2.....	9
10	获取发送功率.....	9
11	获取当前的配置文件信息	10
12	设置当前速率发送功率	12
13	获取当前速率发送功率	12
14	设置信道频点.....	12
15	获取吞吐量.....	13
16	获取信号强度.....	13
17	获取当前连接状态（STA 模式专用）	14
18	获取 WIFI MAC 地址.....	14
19	查询设置状态.....	14
20	设置自适应认证功能	14
21	获取当前的 EFUSE.....	15
22	获取出厂 EFUSE.....	15
23	获取 EFUSE 可用空间.....	15
24	设置 EFUSE 频偏.....	16
25	设置 EFUSE 中的功率因子	16
26	设置 EFUSE MAC 地址.....	17
27	重写 EFUSE 数据.....	17

28	MANAGER 模式下监听 BEACON.....	18
29	清除私有配置设置	18
30	设置最大发包时间	19
31	设置 RTS 门限, TXOP 时间和 TXOP 中 MPDU 个数	19
32	设置 AMPDU 包中最大 MPDU 个数.....	20
33	设置收到 ER SU 包回包模式	20
34	读取温度.....	20
35	获取信道空闲命令	21
36	设置隐藏 SSID.....	21
37	增加 MAC 地址过滤的功能.....	21
38	获取 LMAC MAC 过滤的结果.....	22
39	设置接收 BEACON/PROBE REQ 帧	22
40	设置接收携带自定义 IE 的帧.....	23
41	清除所有的过滤条件	23
42	查询已经设置的过滤条件	23
43	获取 EFUSE 区域的第一个 MAC 地址.....	23
44	设置固定信道扫描	24
45	设置 EDCA 参数.....	24
46	获取连接上设备支持的速率	25
47	设置 POWER SAVE 模式.....	26
48	最佳信道选择【支持 1-13 信道】	27
49	最佳信道选择 2	28
50	支持 IFCONFIG 修改 MAC 地址	28
51	获取当前的发包速率	28
52	设置管理帧速率	29
53	设置单信道扫描时间	29
54	获取芯片内部 PA 状态.....	29
55	获取当前的工作信道	29
56	设置国家码.....	29
57	获取国家码.....	30
58	获取频偏和 DCX0 寄存器.....	30
59	设置 PROBE REQUEST 插入 SSID/密码.....	30

60	监听 PROBE REQUEST 帧.....	31
61	获取 PROBE REQUEST 帧携带的数据.....	31
62	设置 CCA 门限.....	31
63	获取 CCA 门限.....	32
64	获取信噪比.....	32
65	获取底噪.....	32
66	重新加载 FIRMWARE(USB WIFI).....	33
67	连续发送携带私有数据的 PROBE RESP	33
68	停止连续发送携带私有数据的 PROBE RESP	33
69	获取当前的 PA 配置状态.....	33
70	修改 AP 信道.....	34
71	AP 模式下手动设置 TIM.....	34
72	配置 TX 固定速率.....	35
73	ECHO 配置参数命令.....	36
74	设置频偏跟踪使能	37
75	根据温度调节频偏参数修改	38
76	读取寄存器.....	38
77	写寄存器.....	39
78	设计整机校准标识	40
79	整机校准设置 DELTAGAIN.....	40
80	整机校准设置温度补偿	40
81	读取当前 DCX0.....	40
82	设置 DCX0.....	41

作者	版本	说明	时间
Fugui	V1.0	初始版本	20241206
Wangxixi	V1.1	客户版本	20240124
Wangxixi	V1.2	修改 2 条命令： 设置频偏跟踪使能 根据温度调节频偏参数修改 新增 5 条命令： 3 条整机校准，2 条 dcxo	20250217
Wangxixi	V1.3	补充 10 条命令	20250319

1 开始发包命令

命令	<code>iwpriv [interface] start_tx chan,mode,rate,bw,chOff,ldpc,len,[precom]</code>
参数说明	interface: wlan0 或者 p2p0 chan : 信道号, 有效范围[1, 14] mode : 发送模式, 0: 11b; 1: 11a/g; 2: 11n; 3: 11ax HE-SU; 5: 11ax HE-ER_SU rate : 根据发送模式配置对应的速率索引 11b: 0: 1M; 1: 2M; 2: 5.5M; 3: 11M; 11a/g: 0: 6M; 1: 9M;6: 48M; 7: 54M; 11n: 0: mcs0; 7: mcs7; 11ax: 0: mcs0; 11: mcs11; bw : 带宽选择, 0: 20M; 1: 40M; chOff : channel offset, 0: zero; ldpc : 编码方式, 0: BBC; 1: LDPC; 注意: HE-ER_SU 速率只支持 mcs0~mcs2 并且该模式只支持 20M。 HE 发包 20M 带宽 mcs10、mcs11 和 40M 带宽 mcs0-mcs11 必须选中 LDPC 编码。
返回值	成功打印 0, 失败打印-1
例子	<pre>ifconfig wlan0 up 11b 5.5M 发包 iwpriv wlan0 start_tx 1,0,2,0,0,0 11a 54M 发包 iwpriv wlan0 start_tx 1,1,7,0,0,0 11n mcs7 发包 iwpriv wlan0 start_tx 1,2,7,0,0,1 11ax mcs8 发包 iwpriv wlan0 start_tx 1,3,8,0,0,1</pre>

2 停止发包命令

命令	<code>iwpriv [interface] stop_tx</code>
参数说明	interface: wlan0 或者 p2p0
返回值	无
例子	<code>iwpriv wlan0 stop_tx</code>

3 开始收包命令

命令	<code>iwpriv [interface] start_rx chan,bw,chOff,mode</code>
参数说明	interface: wlan0 或者 p2p0

	chan : 信道号, 有效范围[1, 14] bw : 带宽选择, 0: 20M; 1: 40M; chOff : channel offset, 0: zero; 1: Upper; 2: Lower; mode : 0: 接收 ofdm 包; 1: 接收 dsss 包; 2: 接收 ofdm+dsss 包; 注意: 20M 时 chOff 只能设置为 0, 40M 时 chOff 只能设置为 1 或者 2。
返回值	成功打印 0, 失败打印 -1
例子	<pre>ifconfig wlan0 up iwpriv wlan0 start_rx 1,0,0,0</pre>

4 停止收包命令

命令	<code>iwpriv [interface] stop_rx</code>
参数说明	interface: wlan0 或者 p2p0
返回值	成功打印 0, 失败打印 -1
例子	<code>iwpriv wlan0 stop_rx</code>

5 开始发送 singltone 命令

命令	<code>iwpriv [interface] common singltone,<channel></code>
参数说明	interface: wlan0 或者 p2p0 channel : 信道号, [1, 14] 注意: singltone 命令与非信令命令互斥, 发送 singltone 前必须先停止非信令测试
返回值	成功打印 0, 失败打印 -1
例子	<code>iwpriv wlan0 common singltone,7</code>

6 停止 singltone 命令

命令	<code>iwpriv [interface] stop_tx</code>
参数说明	interface: wlan0 或者 p2p0
返回值	成功打印 0, 失败打印 -1
例子	<code>iwpriv wlan0 stop_tx</code>

7 配置发送功率

命令	<code>iwpriv [interface] common</code>
----	--

	<code>set_txpower,<function_index>,<mode>,<rate>,<bw> ,<power></code>
参数说明	interface: wlan0 或者 p2p0 function_index: 0: 代表直接调整 default 功率, 参数: <mode>,<rate>,<bw>,<power_target> 1: 代表按模式增加要给指定的 delta, 参数: <mode>,<power_delta> 2: 代表按带宽的功率增加指定的 delta, 参数: <bw>,<power_delta> mode : 发送模式, 0: 11b; 1: 11a/g; 2: 11n; 3: 11ax rate : 根据发送模式配置对应的速率索引 11b: 0: 1M; 1: 2M; 2: 5.5M; 3: 11M 11a/g: 索引号与速率值相同, 0: 6M; 7: 54M 11n: 0: mcs0; 7: mcs7; 11ax: 0: mcs0; 11: mcs11; bw : 带宽选择, 0: 20M; 1: 40M power_target : 目标功率, 有效范围[-10,20]dBm power_delta : 调整的功率值 (dB)。当前功率调整 power_delta 后仍然应该在功率有效范围内。 注意: 1) 该命令 power 精度只支持到小数十分位。 2) 如果功率增加太多, 比如超过 20dBm, 芯片功耗可能超过 500mA, 建议增加电源驱动能力。
	成功打印 0, 失败打印 -1
例子	<pre>ifconfig wlan0 up 11n mcs7 20M 设置 default 功率 14dBm iwpriv wlan0 common set_txpower,0,2,7,0,14 11b 功率提高 1dB iwpriv wlan0 common set_txpower,1,0,1 20M 带宽所有速率功率降低 2dB iwpriv wlan0 common set_txpower,2,0,-2</pre>

8 设置信号发射功率 1

命令	<code>iwpriv [interface] common set_ladder_txpower,[idx]</code>
参数说明	interface: wlan0 或者 p2p0 idx : 取值 0/3/15/63, 0 表示正常发射功率模式 15 表示超高发射功率模式 63 表示最高发射功率模式 注意: 1) 如果功率增加太多, 比如超过 20dBm, 芯片功耗可能超过 500mA, 建议增加电源驱动

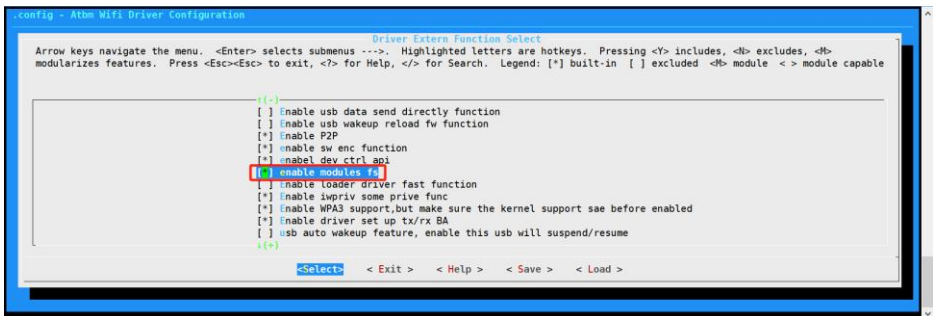
	能力。 2) 该命令是命令 7 的集合，所以两个命令会相互影响。
返回值	成功打印 0，失败打印-1
例子	ifconfig wlan0 up iwpriv wlan0 common set_ladder_txpower,15

9 设置信号发射功率 2

命令	iwpriv [interface] common set_rate_txpower_mode,[idx]
参数说明	interface: wlan0 或者 p2p0 idx : 取值[-16:16] 如果设置 idx 为-16，则发射功率为-8dB。 如果设置 idx 为-6，则发射功率为-3dB。 如果设置 idx 为 0，则发射功率为+0dB。 如果设置 idx 为 6，则发射功率为+3dB。 如果设置 idx 为 16，则发射功率为+8dB。 注意： 1) 如果功率增加太多，比如超过 20dBm，芯片功耗可能超过 500mA，建议增加电源驱动能力。 2) 该命令是命令 7 的集合，所以两个命令会相互影响。
返回值	无
例子	iwpriv wlan0 common set_rate_txpower_mode,-16

10 获取发送功率

命令	1、iwpriv [interface] common get_txpower 2、cat /sys/module/[wifi_module_name]/atbmfs/atbm_get_txpower
参数说明	interface: wlan0 或者 p2p0 注意： 该命令用于获取当前使用的发送功率。 wifi_module_name: 驱动挂载的名字可通过 lsmod 查看，默认的 usb wifi 驱动名称为 ATBM6x6x_wifi_usb 使用该方法需要打开该配置：

	
返回值	成功打印 0，失败打印 -1
例子	方法 1 <pre>ifconfig wlan0 up iwpriv wlan0 common get_txpower</pre> 打印： power:[17]dBm mode:[0]dB bw:[0]dB 方法 2 <pre>cat /sys/module/ATBM6x6x_wifi_usb/atbmfs/atbm_get_txpower</pre> 打印： power:[17]dBm mode:[0]dB bw:[0]dB 结果说明 power 是当前 default 功率【设置目标发射功率的修改的就是这个值】， mode 是按模式调整的功率增益， bw 是按带宽调整的功率增益 芯片实际发送功率为 $current_power = (power + mode + bw)$ 注意： 当前发射功率 current_power 超过 20 请以实际仪器测试为准。

11 获取当前的配置文件信息

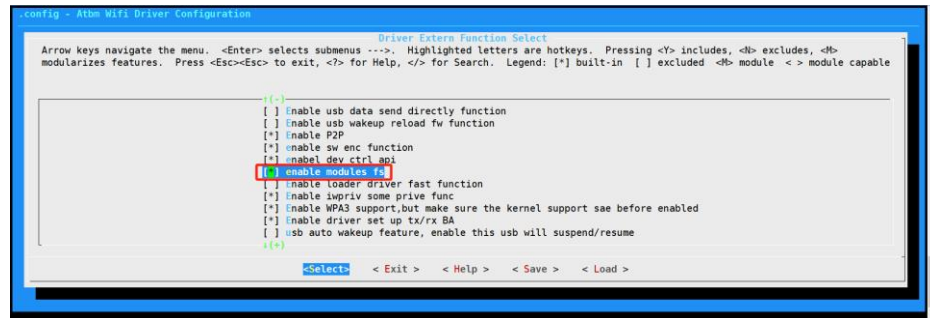
命令	1、iwpriv [interface] common get_cfg_txpower 2、cat /sys/module/[wifi_module_name]/atbmfs/atbm_get_cfg_power
参数说明	interface: wlan0 或者 p2p0 注意：

该命令用于获取当前使用的发送功率。

wifi_module_name:

驱动挂载的名字可通过 `lsmod` 查看，默认的 usb wifi 驱动名称为 `ATBM6x6x_wifi_usb`

使用该方法需要打开该配置：



返回值

成功打印 0，失败打印 -1

方法 1

```
ifconfig wlan0 up
```

```
iwpriv wlan0 common get_cfg_txpower
```

方法 2

```
cat /sys/module/ATBM6x6x_wifi_usb/atbmfs/atbm_get_cfg_power
```

两个方法打印一样：

例子

```
[ALTOBEAM@193] ~$ cat /sys/module/ATBM6x6x_wifi_usb/atbmfs/atbm_get_cfg_power
wlan0 common:b_1M_2M=0
b_5_5M_11M=0
g_6M_n_6_5M=0
g_9M=0
g_12M_n_13M=0
g_18M_n_19_5M=0
g_24M_n_26M=0
g_36M_n_39M=0
g_48M_n_52M=0
g_54M_n_58_5M=0
n_65M=0
HE-SU_MCS0=0
HE-SU_MCS1=0
HE-SU_MCS2=0
HE-SU_MCS3=0
HE-SU_MCS4=0
HE-SU_MCS5=0
HE-SU_MCS6=0
HE-SU_MCS7=0
HE-SU_MCS8=0
HE-SU_MCS9=0
HE-SU_MCS10=0
HE-SU_MCS11=0
g_6M_n_6_5M_40M=0
g_9M_40M=0
g_12M_n_13M_40M=0
g_18M_n_19_5M_40M=0
g_24M_n_26M_40M=0
g_36M_n_39M_40M=0
g_48M_n_52M_40M=0
g_54M_n_58_5M_40M=0
n_65M_40M=0
HE-SU_MCS0_40M=0
HE-SU_MCS1_40M=0
HE-SU_MCS2_40M=0
HE-SU_MCS3_40M=0
HE-SU_MCS4_40M=0
HE-SU_MCS5_40M=0
HE-SU_MCS6_40M=0
HE-SU_MCS7_40M=0
HE-SU_MCS8_40M=0
HE-SU_MCS9_40M=0
HE-SU_MCS10_40M=0
HE-SU_MCS11_40M=0
delta_gain1:27 delta_gain2:27 delta_gain3:27 dcxo:50
```

12 设置当前速率发送功率

命令	<code>iwpriv [interface] common set_power_target,<power_tar></code>
参数说明	<code>interface</code> : wlan0 或者 p2p0 <code>power_target</code>: 目标功率, 有效范围[-10,20]dBm 注意: 该命令用于发包过程中设置当前速率使用的发送功率。
返回值	成功打印 0, 失败打印-1
例子	<pre>ifconfig wlan0 up iwpriv wlan0 common set_power_target,14</pre> 打印: <pre># iwpriv wlan0 common set_power_target,14 [87688.459677] [atbm_log]:cmd:set_power_target,14 , len=20 [87688.466160] [atbm_log]:comma=,14, input_cmd_len = 16 [87688.471608] [atbm_log]:pos:14 [87688.474851] [atbm_log]:powerTar:56,0x38 wlan0 common:set_power_target,14 #</pre>

13 获取当前速率发送功率

命令	<code>iwpriv [interface] common get_power_target</code>
参数说明	<code>interface</code>: wlan0 或者 p2p0 注意: 该命令用于发包过程中获取当前速率使用的发送功率。
返回值	成功打印 0, 失败打印-1
例子	<pre>ifconfig wlan0 up iwpriv wlan0 common get_power_target</pre> 打印: <pre># # iwpriv wlan0 common get_power_target [87739.125872] [atbm_log]:cmd:get_power_target , len=17 [87739.132087] [atbm_log]:powerTar:140/10 wlan0 common:get_power_target #</pre>

14 设置信道频点

命令	<code>iwpriv [interface] fwcmd set_freq,[chan_number],[chan_value]</code> <code>iwpriv [interface] set_freq [chan_number],[chan_value]</code>
----	--

参数说明	interface : wlan0 或者 p2p0 chan_number : 信道号, 取值[1~13] chan_value : 频点参数, 取值 2380 或者 2504, 或者正常的 1~13 频点 注意: 使用命令 2 设置特殊频点可以在文件系统查询到。 cat /sys/modules/atbmxxx/paramaters/wifi_freq
返回值	
例子	1、设置特殊频点 <pre>ifconfig wlan0 up iwpriv wlan0 fwcmd set_freq,1,2380</pre> 2、设置回正常频点 <pre>ifconfig wlan0 up iwpriv wlan0 fwcmd set_freq,1,2412</pre>

注:

(1) 不同的信道不能设置相同的频点, 驱动也需要做这方面的判断, 预防出错。

(2) 正常频点: 2412/2417/2422/2427/2432/2437/2442/2447/2452/2457/2462/2467/2472/2484

(3) 特殊频点: 2380/2504

15 获取吞吐量

命令	iwpriv [interface] get_tp_rate [sta mac addr]
参数说明	interface : wlan0 或者 p2p0 sta mac addr: 设备本身做为 ap 模式需要获取连接的 sta 当前环境的吞吐量, 该参数对应的是 sta 的 mac 地址; sta 模式该参数可以为空
返回值	当前的吞吐量, 值的单位是 bits/s
例子	<pre>ifconfig wlan0 up 连接上需要 ping、或者传输数据才行 iwpriv wlan0 get_tp_rate</pre>

16 获取信号强度

命令	iwpriv [interface] common get_rssi [sta mac addr]
参数说明	interface : wlan0 或者 p2p0 sta mac addr: 设备本身做为 ap 模式需要获取连接的 sta 信号的平均强度, 平均的因子 (alpha 系数) 是 1/8, 该参数对应的是 sta 的 mac 地址; sta 模式该参数可以为空
返回值	当前的 rssi 值
例子	<pre>ifconfig wlan0 up iwpriv wlan0 common get_rssi</pre>

17 获取当前连接状态（sta 模式专用）

命令	<code>iwpriv [interface] get_conn_state</code>
参数说明	interface: wlan0 或者 p2p0
返回值	未连接状态 0, 已连接状态 1
例子	<pre> ifconfig wlan0 up iwpriv wlan0 common get_conn_state # # iwpriv wlan0 common get_conn_state [1845.804546] [atbm_log]:cmd:get_conn_state , len=15 [1845.810378] [atbm_log]:0,DISCONNECTED wlan0 common:get_conn_state # # </pre>

18 获取 wifi mac 地址

命令	<code>iwpriv [interface] getmac</code>
参数说明	interface: wlan0 或者 p2p0
返回值	打印出来 wlan0 或者 p2p0 的 mac 地址: dc:29:19:00:25:7f
例子	<code>iwpriv wlan0 getmac</code>

19 查询设置状态

命令	<code>iwpriv [interface] fwcmd get_cmd_status</code>
参数说明	interface: wlan0 或者 p2p0
返回值	1) 先打印: 当前信道号, 信道带宽, 主边带信道频点, 全信道中心频点。如果带宽是 20M, 则主边带信道频点和全信道中心频点一样; 否则两个值差 10M 或者 -10M。 比如: 6062CU,ch7,20,main2442M,mid2442M 或者: 6062CU,ch7,40+,main2442M,mid2452M 或者: 6062CS,ch7,40-,main2442M,mid2432M 6062CU: USB 芯片 6062CS: SDIO 芯片
例子	<pre> iwpriv wlan0 fwdbg 1 iwpriv wlan0 fwcmd get_cmd_status </pre>

20 设置自适应认证功能

命令	<code>iwpriv wlan0 fwcmd set_adaptive,value</code>
----	--

参数说明	value = 1 开启自适应功能 value = 0 关闭自适应功能
返回值	lmac 有打印: adaptive value
例子	

21 获取当前的 efuse

命令	iwpriv wlan0 common get_efuse
参数说明	无
返回值	Get efuse data is [1,80,5,4,4,6,0,0, dc:29:19:4c:81:e7] 80 : 是dcxo 频偏 6 : 是结温 5,4,4: gain1,gain2,gain3 dc:29:19:4c:81:e7:mac 地址
例子	需要注意在所有设置 efuse 之前都必须先读取出来数据。 所以写 efuse 的流程按照如下规则: 【一定要按照下面的步骤, 不然会设置失败】 1、get efuse 2、set efuse

22 获取出厂 efuse

命令	iwpriv wlan0 common get_first_fuse
参数说明	无
返回值	efuse first data is [1,80,5,4,4,6,0,0, dc:29:19:4c:81:e7] 80 : 是dcxo 频偏 6 : 是结温 5,4,4: gain1,gain2,gain3 dc:29:19:4c:81:e7:mac 地址
例子	iwpriv wlan0 common get_first_fuse

23 获取 efuse 可用空间

命令	iwpriv wlan0 common get_efuse_free
参数说明	无
返回值	efuse free space:[300] bit
例子	iwpriv wlan0 common get_efuse_free

24 设置 efuse 频偏

命令	<code>iwpriv wlan0 common set_efuse_dcxo,<dcxo>,write_rom</code>
参数说明	<code>dcxo</code> : 设置频偏因子的值, 范围是 0~127 <code>write_rom</code>: 是否断电保存数据, 1: 保存; 0: 不保存 通过 <code>get</code> 回来的数据为基准数据: 该值改大, 频偏往小了偏 该值改小, 频偏往大了偏
返回值	无
例子	<code>iwpriv wlan0 common get_efuse</code> 返回值: Get efuse data is [1,80,5,4,4,6,0,0,dc:29:19:4c:81:e7] 当前频偏为: -3 <code>iwpriv wlan0 common set_efuse_dcxo,70,0</code> 当前频偏为: 12 (规律和晶体的曲线相关, 整体呈负相关)

25 设置 efuse 中的功率因子

命令	<code>iwpriv wlan0 common set_efuse_deltagain,<delta_gain1>,<delta_gain2>,<delta_gain3>,write_rom</code>
参数说明	<code>delta_gain1</code>: 低信道功率因子 (0~31) <code>delta_gain2</code>: 中信道功率因子 (0~31) <code>delta_gain3</code>: 高信道功率因子 (0~31) <code>write_rom</code>: 是否断电保存数据。1 保存, 0 不保存 取值 0~15 功率增大 取值 16~31 功率减小 注意: 如果功率增加太多, 比如超过 20dBm, 芯片功耗可能超过 500mA, 建议增加电源驱动能力。
返回值	无
例子	1、芯片内部的 <code>efsue</code> 小于 16 <code>iwpriv wlan0 common get_efuse</code> 返回值: Get efuse data is [1,80,5,4,4,6,0,0,dc:29:19:4c:81:e7] <code>iwpriv wlan0 common set_efuse_deltagain,9,8,8,1</code> 功率因子修改前后的功率差别: 低信道功率增加: $(9-5)/4 = 1\text{dB}$ 中信道功率增加: $(8-4)/4 = 1\text{dB}$ 高信道功率增加: $(8-4)/4 = 1\text{dB}$ <code>iwpriv wlan0 common set_efuse_deltagain,17,18,19,1</code> 功率因子修改后的功率差别:

	低信道功率减少: $(17-32)/4 - 5/4 = -5\text{dB}$ 中信道功率减少: $(18-32)/4 - 4/4 = -4.5\text{dB}$ 高信道功率减少: $(19-32)/4 - 4/4 = -4.25\text{dB}$ 2、芯片内部的 efsue 大于 16 <code>iwpriv wlan0 common get_efuse</code> 返回值: Get efuse data is [1,80,25,24,24,6,0,0,dc:29:19:4c:81:e7] <code>iwpriv wlan0 common set_efuse_deltagain,9,8,8,1</code> 功率因子修改前后的功率差别: 低信道功率增加: $9/4 - (25 - 32)/4$ 中信道功率增加: $8/4 - (24 - 32)/4$ 高信道功率增加: $8/4 - (24 - 32)/4$ <code>iwpriv wlan0 common set_efuse_deltagain,17,18,19,1</code> 功率因子修改后的功率差别: 低信道功率减少: $(17-32)/4 - (25 - 32)/4$ 中信道功率减少: $(18-32)/4 - (24 - 32)/4$ 高信道功率减少: $(19-32)/4 - (24 - 32)/4$
--	---

26 设置 efuse mac 地址

命令	<code>iwpriv wlan0 common set_efuse_mac,<mac 地址></code>
参数说明	mac 地址的格式必须是 xx:xx:xx:xx:xx:xx
返回值	无
例子	<code>iwpriv wlan0 common get_efuse</code> 返回值: Get efuse data is [1,80,5,4,4,6,0,0,dc:29:19:4c:81:e7] <code>iwpriv wlan0 common set_efuse_mac,10:02:03:04:05:01</code> 修改完成不会立即生效，需要立即生效需要重新卸载/加载下驱动。

27 重写 efuse 数据

命令	<code>iwpriv wlan0 common set_all_efuse,dcxo,gain1,gain2,gain3,tjroom,mac</code>
参数说明	dcxo: 频偏因子, 有效范围是[0, 127] gain1: 低信道功率因子 (0~31) gain2: 中信道功率因子 (0~31) gain3: 高信道功率因子 (0~31) tjroom: 校准时获取的结温 mac 地址: 格式必须是 xx:xx:xx:xx:xx:xx

返回值	无
例子	<pre>iwpriv wlan0 common get_efuse</pre> 返回值: Get efuse data is [1,80,5,4,4,6,0,0,dc:29:19:4c:81:e7] <pre>iwpriv wlan0 common set_all_efuse,75,31,31,31,8,10:02:03:04:05:01</pre> 修改完成不会立即生效，需要立即生效需要重新卸载/加载下驱动。

28 Manager 模式下监听 beacon

命令	<code>iwpriv wlan0 sta_channel <channel></code>
参数说明	channel: 信道，取值范围为[1,14] 注意：打开宏 CONFIG_ATBM_STA_LISTEN
返回值	无
例子	监听 1 信道的 beacon <pre>ifconfig wlan0 up</pre> <pre>iwpriv wlan0 sta_channel 1</pre>

29 清除私有配置设置

命令	<code>iwpriv wlan0 ipc_reset <paramaters></code>
参数说明	paramaters : - data: 清空插入的私有数据 - mac : 清空 mac 地址过滤功能 - recv: 清空接收过滤回来的数据 - all : 清空所有的配置 注意：打开宏 ATBM_PRIVATE_IE
返回值	无
例子	1、清空插入的私有数据 <pre>ifconfig wlan0 up</pre> <pre>iwpriv wlan0 ipc_reset data</pre> 2、清空过滤的 mac 地址 <pre>ifconfig wlan0 up</pre> <pre>iwpriv wlan0 ipc_reset mac</pre> 3、清空过滤回来的所有数据 <pre>ifconfig wlan0 up</pre> <pre>iwpriv wlan0 ipc_reset recv</pre>

	4、清空所有配置以及数据 ifconfig wlan0 up iwpriv wlan0 ipc_reset all
--	---

30 设置最大发包时间

指令	iwpriv wlan0 fwcmd txttime,params
参数	params: 单位是 ms, 值只能在 100~10*1024 这个范围, 否则不生效
功能描述	设置包最大发送时间, 默认是 2000ms
返回值	需要先打开 iwpriv wlan0 fwdbg 1, 后在进行设置会看到打印 LiveTime%d,%d: 第一个参数打印输入参数, 第二个参数打印表示实际生效值
指令示例	iwpriv wlan0 fwcmd txttime,128: 最大发包时间设置 128ms iwpriv wlan0 fwcmd txttime,5000: 最大发包时间设置 5000ms

31 设置 rts 门限, txop 时间和 txop 中 mpdu 个数

指令	iwpriv wlan0 fwcmd rts,get_or_set,rts_time_thr,rts_duration,mpdu_num, rts_cf_end
参数	get_or_set : 0 打印该命令设置情况, 具体见下面返回值; 1 设置该命令。 rts_time_thr: 只有参数 get_or_set 为 1 才生效, 设置发 rts 包的时间门限, 单位是 us, 该参数设置不能超过 10000 (对应 10ms), 如果超过就限制成 10000; 该参数设置为 0 的时候, rts 门限就是和 ap 协商的。目前 IPC 固件该参数默认是 500, 也就是只要组包长度超过 500us 就会发 rts 包; 电视固件该参数是和 AP 协商出来的。 rts_duration: 只有参数 get_or_set 为 1 才生效, 设置 rts 保护间隔, 也叫做 txop 时间, 单位是 us。该参数不能超过 8191, 如果超过就限制成 8191。如果该参数是 0, 用的是 ap 协商的时间; 如果该参数小于 500, 用默认的 txop 时间 (默认是动态变化的, 最小 2000us, 最大 8000us); 该参数大等于 500, 才生效用于 txop 时间。 mpdu_num : 只有参数 get_or_set 为 1 才生效, 表示设置 txop 时间内最多元勋发送 mpdu 包个数, 最大不能超过 63 个。如果该参数是 0, 用的是默认值 (63)。 rts_cf_end : 是否发 cf end, IPC 固件默认是 1, 电视固件默认是 0。
功能描述	设置 rts 门限, txop 时间和 txop 中 mpdu 个数。
返回值	RTS:Thr%dus,Txop%dus,MPDU%d,CFEnd%d
指令示例	获取 rts 命令设置情况, 会打印上面返回值: iwpriv wlan0 fwcmd rts,0 设置发 rts 包时间是 500us, rts duration 和 mpdu 个数用默认参数, 发送 CF END 包: iwpriv wlan0 fwcmd rts,1,500,1,0,1 设置发 rts 包时间是 1000us, rts duration 为 3000us, mpdu 个数为 32, 不发送 CF

	END 包: iwpriv wlan0 fwcmd rts,1,1000,3000,32,0
--	---

32 设置 ampdu 包中最大 mpdu 个数

指令	iwpriv wlan0 fwcmd ampdu,num
参数	num: 只能取 1~64, 超过该值用默认的 20
功能描述	设置一次组 ampdu 最多 mpdu 个数, 注意这个参数是针对包长度是 1538byte 的个数, 如果是跑 tx, 发送包长度一般是 1538byte, 那么每次聚合 ampdu 最多 mpdu 个数是 num; 但如果跑 tcp rx, 一般 tcp ack 包长度是 90byte, 那么实际组 ampdu 最多 mpdu 个数 $1538 * num / 90$
返回值	AMPDUMaxNum: %d
指令示例	设置每次组 ampdu 最大 mpdu 的个数是 8 个: iwpriv wlan0 fwcmd ampdu,8

33 设置收到 ER SU 包回包模式

指令	iwpriv wlan0 fwcmd ersu,mode
参数	mode: 只能取 0 或 1, 默认是 0
功能描述	设置收到 ER SU 包回的响应包 (CTS/ACK/BA) 是 legacy 6M 包还是 ER SU 包
返回值	ERSU %d
指令示例	收到 ER SU 包回响应包是 legacy 包: iwpriv wlan0 fwcmd ersu,0 收到 ER SU 包回响应包是 ER SU 包: iwpriv wlan0 fwcmd ersu,1

34 读取温度

命令	iwpriv wlan0 get_tjroom
参数说明	获取当前芯片内部温度
返回值	无
例子	iwpriv wlan0 common get_tjroom ~ # iwpriv wlan0 common get_tjroom [39160.979102] [atbm_log]:cmd:get_tjroom , len=11 [39160.985065] [atbm_log]:Tjroom:9, tempc:542,stempc:43

35 获取信道空闲命令

命令	<code>iwpriv wlan0 common get_channel_idle</code>
参数说明	
返回值	<code>current_idle:value</code>，其中信道空闲 <code>value</code> 取值范围是 1~1024。 值越大，信道越空闲，信道空闲测量底层是每 2~3s 才计算一次。 如果信道空闲值小于 100，建议换信道或者降低码率。 当打印信道空闲值为 0 时，表示信道测量还没测出来，可以 3s 后再测量一次。 当读取信道空闲值和上次值一样，表示当前的信道空闲还没计算出来，这个时候打印的还是上次测量的信道空闲值 注意在跑吞吐率或 ping 结束后输入这个命令显示最后统计的空闲比 由于一次信道空闲值可能不准，建议读取多次信道空闲值做平均，注意做平均时需要去掉和上次信道空闲值一样的值。 比如： 多次读取信道空闲值时： <code>current_idle:100,</code> <code>current_idle:200,</code> <code>current_idle:200,</code> <code>current_idle:150,</code> <code>current_idle:200,</code> <code>current_idle:100,</code> 平均出来值应该是 $(100+200+150+200+100)/5=150$， 而不是 $(100+200+200+150+200+100)/6=950/6$
例子	<code>iwpriv wlan0 common get_channel_idle</code>

36 设置隐藏 ssid

命令	<code>iwpriv wlan0 common ap_hide_ssid,<enable></code>
参数说明	该命令必须要在已经启动了 AP 模式才有效果 <code>enable: 1</code> 隐藏 ssid, <code>0</code> 关闭隐藏 ssid 功能
返回值	成功返回 0，失败返回 -1
例子	

37 增加 mac 地址过滤的功能

命令	<code>iwpriv wlan0 common monitor_mac,switch,mac_addr</code>
参数说明	<code>switch : 1</code> 打开 lmac mac 过滤, <code>0</code> 关闭 lmac_mac 过滤 <code>mac_addr</code>: 需要过滤的 mac 地址
返回值	成功返回 0，失败返回 -1
例子	1、打开 lmac mac 过滤功能

	先打开 lmac 打印: <code>iwpriv wlan0 fwdbg 1</code>, 在执行命令确认有设置到 lmac <pre>iwpriv wlan0 common monitor_mac,1,11:22:33:44:55:66 [lmac]:monitor,staDC,29,19,4C,81,E7</pre> 2、关闭 lmac mac 过滤功能 <pre>iwpriv wlan0 common monitor_mac,0</pre>
--	---

38 获取 lmac mac 过滤的结果

命令	<code>iwpriv wlan0 common monitor_status</code>
参数说明	无
返回值	成功返回 0, 失败返回 -1
例子	<pre>[/ext/demo]## iwpriv wlan0 common monitor_status wlan0 common:found=[1],rssi=[-23],delta_time(1996)ms [/ext/demo]## iwpriv wlan0 common monitor_status wlan0 common:found=[1],rssi=[-23],delta_time(3108)ms [/ext/demo]## iwpriv wlan0 common monitor_status wlan0 common:found=[1],rssi=[-22],delta_time(32)ms ---- 32ms 前收到这个包 [/ext/demo]## iwpriv wlan0 common monitor_status wlan0 common:found=[1],rssi=[-22],delta_time(1024)ms</pre> 结果说明: found 代表收到 mac 地址对应的设备, 只要收到就一直为 1, 直到关闭 mac 过滤 rssi 设备的信号强度 delta_time 距离上一次收到这个设备无线包的时间。

39 设置接收 beacon/probe req 帧

命令	<code>iwpriv wlan0 common filter_frame,<idx></code>
参数说明	idx : 80 接收 beacon 帧 40 接收 probe req 帧 需要启动 hostapd 或者 wpa_supplicant
返回值	成功返回 0, 失败返回 -1
例子	1、设置过滤 beacon 帧 <pre>iwpriv wlan0 common filter_frame,80</pre> 2、设置过滤 probe req 帧 <pre>iwpriv wlan0 common filter_frame,40</pre>

	数据处理函数 hal_apollo/mac80211/special_filter.c ieee80211_special_filter_rx_package_handle
--	---

40 设置接收携带自定义 ie 的帧

命令	iwpriv wlan0 common filter_ie,<id>,<oui1>,<oui2>,<oui3>
参数说明	id:IE 对应的 id 号,十进制 oui1-oui2-oui3:IE 对应的 oui, 十进制 oui1,oui2,oui3 可为空 需要启动 hostapd 或者 wpa_supplicant
返回值	成功返回 0, 失败返回-1
例子	1、设置过滤 221 字段的帧 iwpriv wlan0 common filter_ie,221 2、设置过滤 221 字段 oui 为 5-6-7 的帧 iwpriv wlan0 common filter_ie,221,5,6,7 数据处理函数 hal_apollo/mac80211/special_filter.c ieee80211_special_filter_rx_package_handle

41 清除所有的过滤条件

命令	iwpriv wlan0 common filter_clear
参数说明	无
返回值	成功返回 0, 失败返回-1
例子	需要注意如果设置使用的网口是 p2p0, 那么清除也需要使用 p2p0 网口进行清除数据

42 查询已经设置的过滤条件

命令	iwpriv wlan0 common filter_show
参数说明	无
返回值	成功返回 0, 失败返回-1
例子	

43 获取 efuse 区域的第一个 mac 地址

命令	iwpriv wlan0 common get_first_mac
----	-----------------------------------

参数说明	无
返回值	成功返回 0，失败返回 -1
例子	

44 设置固定信道扫描

命令	<code>iwpriv wlan0 common ap_conf,channel</code>
参数说明	驱动需要打开宏 <code>CONFIG_ATBM_SUPPORT_AP_CONFIG</code> channel: 1~14
返回值	成功返回 0，失败返回 -1
例子	1、设置固定 1 信道扫描 <pre>iwpriv wlan0 common ap_conf,3 iwlist wlan0 scan</pre> 此时只会扫描 3 信道 2、关闭固定信道扫描 <pre>iwpriv wlan0 common ap_conf,0 iwlist wlan0 scan</pre> 此时会扫描 1~14 信道

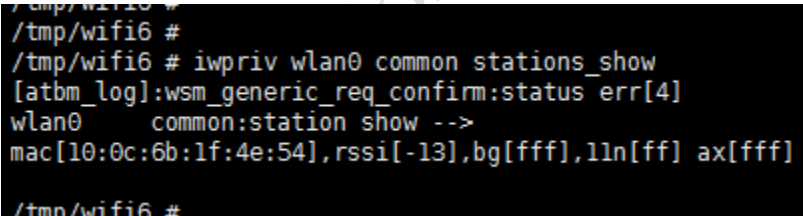
45 设置 edca 参数

命令	<code>iwpriv wlan0 common edca_params,queue,aifs,cw_min,cw_max,txop</code>
参数说明	queue: 0: voice traffic 1: video traffic 2: best effort 3: background traffic txop: 传输机会, 参数值为 TXOPlimit, 代表占用信道发包最长时间, 取值范围: 0~65535, 取值为 0 代表只发送一个 MPDU. 如果当前 txop 设置为 0 时, 除非是专门认证的固件, 否则 txop 会更加当前使用的速率自动计算出一个合适的值, txop 设置的值就是 rts_duration 值, 不同的发包优先级对应的有不同的 rts_duration。 cw_min: 最小竞争窗口, 越小的 CWmin 其优先级越高, 取值范围: 2~15 cw_max: 最大竞争窗口, 越小的 CWmax 其优先级越高, CWmax > CWmin, 取值范围: 3~1023 aifs: EDCA 模式的 QSTA 要获得传输机会时, 必须等待的信道空闲时间, 时间越大代表优先级越低, 取值范围: 2~15 注意: sta 这边配置了一套 edca 参数, 如果该 sta 连上 ap, 那么 sta 这边原本配置的 edca 参数会被刷新为 ap 的 edca 参数, 连接上 ap 以后重新设置会再次生效。 抓包看变化不明显, 主要在本地会生效, 自己的发包策略会变化。

返回值	set 命令 成功返回 0，失败返回-1 get 命令 [atbm_log]:[voice_traffic] [atbm_log]:aifns : 1 [atbm_log]:cwMax : 7 [atbm_log]:cwMin : 3 [atbm_log]:txOpLimit : 1504 [atbm_log]:[video_traffic] [atbm_log]:aifns : 1 [atbm_log]:cwMax : 15 [atbm_log]:cwMin : 7 [atbm_log]:txOpLimit : 3008 [atbm_log]:[best_effort] [atbm_log]:aifns : 3 [atbm_log]:cwMax : 63 [atbm_log]:cwMin : 15 [atbm_log]:txOpLimit : 0 [atbm_log]:[background_traffic] [atbm_log]:aifns : 7 [atbm_log]:cwMax : 1023 [atbm_log]:cwMin : 15 [atbm_log]:txOpLimit : 0
例子	set 命令 iwpriv wlan0 common edca_params,0,2,3,255,100 get 命令 iwpriv wlan0 common get_edca_params

46 获取连接上设备支持的速率

命令	iwpriv wlan0 common stations_show
参数说明	无
返回值	成功返回 0，失败返回-1

例子	<pre> ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-77],bg[fff],11n[ff] ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-76],bg[fff],11n[ff] ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-76],bg[fff],11n[ff] ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-76],bg[fff],11n[ff] ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-75],bg[fff],11n[ff] </pre>
	说明: mac : 连接设备的 mac 地址 rssi : 信号强度 bg/11n 协商的速率: 其中: bg[fff] --> 转成二进制共 12 位对应 b/g 12 个速率: 从低到高对应速率: [1M 2M 5.5M 11M 6M 9M 12M 18M 24M 36M 48M 54M] 如上图: bg[fff] 代表支持 b/g 全速率 如果 : bg[55f] 代表支持 [1M 2M 5.5M 11M 6M 12M 24M 48M] 其中 : 11n[ff] 从低到高对应的是 MCS0~MCS7 如上图 : 11n[ff] 代表当前连接支持 MCS0~MCS7 如果 : 11n[53] 代表支持 [MCS0 MCS1 MCS4 MCS6]
	
	如果连接的路由器支持 WIFI6 会增加 ax[%x] 打印: ax[fff] => 支持 MCS0~MCS11 如果是 ax[1ff] => 0001 1111 1111 => 支持 MCS0~MCS8

47 设置 power save 模式

命令	iwpriv wlan0 common power_save,value
参数说明	value: 1: 开启休眠功能 0: 关闭休眠功能 进入休眠以后会发 null data 包, 包里面 power save 位为 1
返回值	成功返回 0, 失败返回 -1
例子	

48 最佳信道选择【支持 1-13 信道】

命令	<code>iwpriv wlan0 best_ch_scan,start_ch,end_ch,specialFlag</code>
参数说明	有特殊信道使用需要打开宏: <code>CONFIG_ATBM_5G_PRETEND_2G</code> 参数说明: <code>start_channel</code>: 起始统计信道 <code>end_channel</code> : 终止统计信道 <code>specialFlag</code> : 是否统计特殊信道
返回值	成功返回 0, 失败返回-1
例子	建议使用自带的 <code>iwpriv_demo</code> 进行测试: 1、<code>iwpriv wlan0 best_ch_scan</code> 默认统计 1~14 信道的最佳信道, 但是应用层获取不到数据, 内核只打印最佳信道 2、<code>iwpriv wlan0 best_ch_scan,start_ch,end_ch specialFlag</code> 会推荐三个最优信道, 最佳信道, 次佳信道, 次次佳信道 2.1) <code>iwpriv wlan0 best_ch_scan 1,11,0</code> 统计 1~11 信道 *注:<code>iwpriv_demo</code> 非开源 <code>iwpriv</code> 命令。 2.2) <code>iwpriv_demo wlan0 best_ch_scan 1 14 0</code> 统计 1~14 信道 2.3) <code>iwpriv_demo wlan0 best_ch_scan 1 14 1</code> 统计 1~14 信道 2.4) <code>iwpriv_demo wlan0 best_ch_scan 1 11 1</code> 统计 1~11 信道 2.5) <code>iwpriv_demo wlan0 best_ch_scan 1 42 0</code> 统计 1~14 信道 2.6) <code>iwpriv_demo wlan0 best_ch_scan 1 42 1</code> 统计 1~14 信道 + 统计 36/38/40/42 信道 2.7) <code>iwpriv_demo wlan0 best_ch_scan 1 36 1</code> 统计 1~14 信道+统计 36 信道 2.8) <code>iwpriv_demo wlan0 best_ch_scan 1 36 0</code> 统计 1~14 信道

49 最佳信道选择 2

命令	<code>iwpriv wlan0 common best_ch_scan2,start_chan,end_chan,special,ssid</code>
参数说明	<code>start_chan</code>: 起始信道: [1,14] <code>end_chan</code>: 终止信道: [1,14] 必须要比起始信道号大 1 以上 <code>special</code>: 备用参数 <code>ssid</code>: 统计所有信道中的同名 <code>ssid</code> 的 AP 信息, 共计保存 14 个
功能描述	增加统计所有信道中的同名的 <code>ssid</code> 的 ap 信息, 必须是 4 个参数
返回值	成功返回 0, 失败返回 -1
例子	<code>iwpriv wlan0 common best_ch_scan2,1,14,0,yzh_test</code>

50 支持 ifconfig 修改 mac 地址

命令	<code>ifconfig wlan0 hw ether [mac_addr]</code>
参数说明	1. wlan0/1 网口状态必须处于 down 状态 2. 只能修改 wlan0 网口的 mac 地址。 3. 打开宏 <code>ccflags-y += -DCONFIG_ATBM_SUPPORT_REALTIME_CHANGE_MAC</code>
返回值	成功返回 0, 失败返回 -1
例子	<code>ifconfig wlan0 hw ether 00:11:22:33:44:55</code>

51 获取当前的发包速率

命令	<code>iwpriv wlan0 common get_maxrate[,mac_addr]</code>
参数说明	<code>mac_addr</code>: wlan0 做为 AP 模式时候使用的参数, <code>mac_addr</code> 为连接上 AP 的 sta mac 地址
返回值	成功返回 0, 失败返回 -1
例子	1、sta 模式获取当前的主要发包速率 <code>iwpriv wlan0 common get_maxrate</code> 2、ap 模式获取与连接的 sta (00:11:22:33:44:55) 当前的主要发包速率 <code>iwpriv wlan0 common get_maxrate,00:11:22:33:44:55</code>

52 设置管理帧速率

命令	<code>iwpriv wlan0 common set_frame_rate,value</code>
参数说明	在 NVR IPC 固件，默认将管理帧的速率设置为 2Mbps value: 1: 设置管理帧为 1Mbps 0: 恢复为 2Mbps 速率
返回值	
例子	

53 设置单信道扫描时间

命令	<code>iwpriv wlan0 common set_single_scan_time,time</code>
参数说明	time: 取值范围[36,255]
功能描述	设置单信道扫描的最大时间，多信道不生效
返回值	0 设置成功，其他设置失败
例子	

54 获取芯片内部 PA 状态

命令	<code>iwpriv wlan0 common get_pa_state</code>
参数说明	无
功能描述	获取芯片内部 PA 的工作状态，使用外置 PA 需要关闭芯片内部 PA，通过该命令来查询当前固件使用是否正确
返回值	0 设置成功，其他设置失败
例子	

55 获取当前的工作信道

命令	<code>iwpriv wlan0 common get_work_channel</code>
参数说明	无
返回值	
例子	

56 设置国家码

命令	<code>iwpriv wlan0 common country_code,[country]</code>
----	---

参数说明	country: 只支持如下三个参数 CN: 中国, 支持 1~13 信道 US: 美国, 支持 1~11 信道 JP: 日本, 支持 1~14 信道
返回值	
例子	设置国家码为美国 ifconfig wlan0 up iwpriv wlan0 common country_code,US

57 获取国家码

命令	iwpriv wlan0 common get_country_code
参数说明	country: 只支持如下三个参数 CN: 中国, 支持 1~13 信道 US: 美国, 支持 1~11 信道 JP: 日本, 支持 1~14 信道
返回值	
例子	设置国家码为美国 ifconfig wlan0 up iwpriv wlan0 common get_country_code

58 获取频偏和 dcxo 寄存器

命令	iwpriv wlan0 common get_cali_data
参数说明	无
返回值	
例子	iwpriv wlan0 common get_cali_data

59 设置 probe request 插入 ssid/密码

命令	iwpriv wlan0 common user_vendor_ie,ssid,psk
参数说明	ssid: 可连接的 ap 的 ssid psk : 可连接的 ap 的密码 注意: wlan0 只能是 sta 模式
返回值	
例子	iwpriv wlan0 common user_vendor_ie,yzh_test,12345678 抓空中包能看到发送的 probe req 携带私有数据, 私有 IE 为 221

60 监听 probe request 帧

命令	<code>iwpriv wlan0 common listen_probe_req,channel</code>
参数说明	<code>channel</code>: 需要监听的信道，如果 <code>sta</code> 已经连接上 <code>ap</code>，那么监听信道只能在 <code>ap</code> 所在的信道，<code>channel</code> 参数无效 注意： <code>wlan0</code> 只能是 <code>sta</code> 模式
返回值	
例子	<code>iwpriv wlan0 common listen_probe_req,1</code> 在 1 信道监听接收 probe req 帧

61 获取 probe request 帧携带的数据

命令	<code>iwpriv wlan0 common get_user_vendor_ie</code>
参数说明	无
返回值	
例子	<code>iwpriv wlan0 common get_user_vendor_ie</code> 如果有监听到数据： <code>atbm_ioctl_get_vendor_ie : ssid[yzh_test],password[12345678]</code> 如果没有数据： <code>atbm_ioctl_get_vendor_ie : ssid[],password[]</code>

62 设置 cca 门限

指令	<code>iwpriv wlan0 fwcmd cca,<set_or_get>,<set_cca_thr></code>
参数	<code><set_or_get></code>: 0: 获取当前信道忙门限（默认是-75dBm）；1: 设置当前信道忙门限 <code><set_cca_thr></code>: 该参数只能设置[-85, -60]dB，超过这个范围参数设置无效，也就是用上次设置值 <code>set_cca_thr</code>（没有设置就用默认值-75dbm）；物理意义就是在能量检测 cca 门限值。 注意： 1) 我们芯片检测信道空闲或者繁忙有两个方法，收到一个包（包检测方法）或者检测到能量比较大（能量检测方法）都会产生信道忙，只有没收到包且能量小才会产生信道空闲。 2) 这里设置 cca 门限只是设置能量检测的门限，值越大（比如最大-60dBm）能量检测越容易出信道空闲；如果在空气中有很多包或者在跑吞吐率，则设这 cca 门限基本没用，因为收到包就触发信道忙，根本不走能量检测逻辑。

功能描述	设置 cca 门限。
返回值	打印: ccadBm %d %d: 第一个参数表示设置值, 第二个参数表示实际生效值, 如果第一个参数在[-85, -60]区间, 第二个参数和第一个参数一样, 否则不生效第二个参数就打印上次生效值。
指令示例	设置信道忙门限为-70dbm iwpriv wlan0 fwcmd cca,1,-70 获取信道忙门限 iwpriv wlan0 fwcmd cca,0

63 获取 cca 门限

指令	iwpriv wlan0 common get_cca_threshold
参数	无
功能描述	获取 CCA 门限。将门限数据上传到驱动。
返回值	打印 : cca_threshold:%d 失败打印: cca_threshold get fail
指令示例	iwpriv wlan0 common get_cca_threshold cca_threshold:-75

64 获取信噪比

指令	iwpriv wlan0 common get_snr
参数	无
功能描述	获取设置的 rts duration 值
返回值	获取成功打印: lmac get snr_val = %d ,real snr = 10*log10(4096/%d) 失败打印 : snr get fail WIFI4 中有效数据获取出来的 snr 值需要在应用端进行计算出实际有效的 snr: snr = 10*log10(4096/ snr_val) WIFI6 中该参数结果为 EVM
指令示例	iwpriv wlan0 common get_snr

65 获取底噪

指令	iwpriv wlan0 common get_noise_level
参数	无
功能描述	获取底噪
返回值	打印: 1) 底噪有效打印: exact noise_level_dBm = [%d/4]

	2) 底噪是无效数据的打印: <code>initg = [%d/4],reckon noise_level_dBm = [%d/4]</code> 3) 失败打印: <code>noise get fail</code>
指令示例	<code>iwpriv wlan0 common get_noise_level</code>

66 重新加载 firmware(USB wifi)

指令	<code>iwpriv wlan0 common fw_reset,wifi_ble_comb</code>
参数	<code>wifi_ble_comb</code> : 1: 重新加载 wifi ble 共存固件 0: 重新加载独立的 wifi 固件
功能描述	重新加载 firmware
返回值	返回 0
指令示例	<code>iwpriv wlan0 common fw_reset,1</code>

67 连续发送携带私有数据的 probe resp

指令	<code>iwpriv wlan0 common send_probe_resp,ssid,psk</code>
参数	<code>ssid</code> : ap 要广播的 ssid <code>psk</code> : ap 的密码
功能描述	ap 模式下使用有效, 在 ap 模式下以间隔为 5ms 的时间一直发送携带私有数据的 probe resp 包
返回值	成功返回 0, 失败非 0
指令示例	Hostapd.conf 配置的 ssid 和 psk 为: yzh_test 12345678 <code>iwpriv wlan0 common send_probe_resp,yzh_test,12345678</code>

68 停止连续发送携带私有数据的 probe resp

指令	<code>iwpriv wlan0 common stop_probe_resp</code>
参数	无
功能描述	AP 模式下使用, 停止连续发送携带私有数据的 probe resp
返回值	成功返回 0, 失败返回非 0
指令示例	<code>iwpriv wlan0 common stop_probe_resp</code>

69 获取当前的 PA 配置状态

指令	<code>iwpriv wlan0 common get_pa_state</code>
参数	无

功能描述	获取当前芯片是否是 FEM 的配置。 如果是 FEM 的配置会打印: PA on 否则打印 PA off
返回值	成功返回 0, 失败返回非 0
指令示例	<code>iwpriv wlan0 common get_pa_state</code>

70 修改 ap 信道

指令	<code>iwpriv wlan0 common change_chan,chan,ht40</code>
参数	chan: 信道范围[1,14] ht40: 是否是 HT40 模式, 0: 非 HT40; 1: HT40 模式
功能描述	AP 模式下使用, 不用重启 hostapd 实现切换信道, 但必须使用高拓提供的 hostapd 才行 hostapd -v 显示为: hostapd_xx_ATBM_XX_XXXXXX
返回值	成功返回 0, 失败返回非 0
指令示例	切换到 1 信道 HT20 模式: <code>iwpriv wlan0 common change_chan,1,0</code> 切换到 1 信道 HT40 模式: <code>iwpriv wlan0 common change_chan,1,1</code>

71 Ap 模式下手动设置 TIM

指令	<code>iwpriv wlan0 common set_tim,ena,val</code>
参数	ena: 0 : 关闭手动模式, 开启自动模式 1 : 开启手动模式, 关闭自动模式 val: 十六进制方式设置 Bitmap control & Partial Virtual Bitmap 的值, 其中低 8bit 为 Bitmap control 控制位, 可以设置为 0/1。AltoBeam wifi Partial Virtual Bitmap 有 2 字节共 24bit 控制位, 当连接的 sta 个数少于 8 个, 可以控制的只有 8bit, 当连接的 sta 个数超过 8 个, 可以控制的有 16bit。
功能描述	AP 模式下使用
返回值	成功返回 0, 失败返回非 0
指令示例	将连接上 ap 的第 1 个 sta 和第 3 个 sta 的 TIM bitMap 置位 <code>iwpriv wlan0 common set_tim,1,000a00</code> b7-----b0 0a =====> 0000 1010 Bit0 对应的是 aid0, 该 bit 在协议上为保留位不使用。

```
/tmp/bin # iwpriv p2p0 common set_tim,1,000a00
[atbm_log]:cmd:set_tim,1,000a00 , len=17
[atbm_log]:comma=,1,000a00, input_cmd_len = 7
[atbm_log]:val set,set_tim,1,000a00
[atbm_log]:ena=1 , tim_bit=000a00
[atbm_log]:get data : ena:1, val:a
p2p0      common:
set_tim success
```

将连接上 ap 的第 1 个 sta 和第 3 个 sta 的 TIM bitMap 置位,并且将 bitmap control 置 1

```
iwpriv wlan0 common set_tim,1,000a01
```

```
/tmp/bin # iwpriv p2p0 common set_tim,1,000a01
[atbm_log]:cmd:set_tim,1,000a01 , len=17
[atbm_log]:comma=,1,000a01, input_cmd_len = 7
[atbm_log]:val set,set_tim,1,000a01
[atbm_log]:ena=1 , tim_bit=000a01
[atbm_log]:get data : ena:1, val:b
p2p0      common:
set_tim success
```

72 配置 tx 固定速率

指令	iwpriv wlan0 common tx_rate,xx	
参数	xx: 固定 TX 速率的速率值范围为[0,37]	
功能描述	固定速率表最高值 37，最低值是 0，配置 255 恢复自动速率 <pre> #define RATE_INDEX_B_1M 0 #define RATE_INDEX_B_2M 1 #define RATE_INDEX_B_5_5M 2 #define RATE_INDEX_B_11M 3 #define RATE_INDEX_PBCC_22M 4 // not supported/unused #define RATE_INDEX_PBCC_33M 5 // not supported/unused #define RATE_INDEX_A_6M 6 #define RATE_INDEX_A_9M 7 #define RATE_INDEX_A_12M 8 #define RATE_INDEX_A_18M 9 #define RATE_INDEX_A_24M 10 #define RATE_INDEX_A_36M 11 #define RATE_INDEX_A_48M 12 #define RATE_INDEX_A_54M 13 #define RATE_INDEX_N_6_5M 14 #define RATE_INDEX_N_13M 15 #define RATE_INDEX_N_19_5M 16 #define RATE_INDEX_N_26M 17 </pre>	

	<pre> #define RATE_INDEX_N_39M 18 #define RATE_INDEX_N_52M 19 #define RATE_INDEX_N_58_5M 20 #define RATE_INDEX_N_65M 21 #define RATE_INDEX_HE_MCSER0 23 #define RATE_INDEX_HE_MCSER1 24 #define RATE_INDEX_HE_MCSER2 25 #define RATE_INDEX_HE_MCS0 26 #define RATE_INDEX_HE_MCS1 27 #define RATE_INDEX_HE_MCS2 28 #define RATE_INDEX_HE_MCS3 29 #define RATE_INDEX_HE_MCS4 30 #define RATE_INDEX_HE_MCS5 31 #define RATE_INDEX_HE_MCS6 32 #define RATE_INDEX_HE_MCS7 33 #define RATE_INDEX_HE_MCS8 34 #define RATE_INDEX_HE_MCS9 35 #define RATE_INDEX_HE_MCS10 36 #define RATE_INDEX_HE_MCS11 37 37 表示 HE 模式 MCS11， 0 表示 11B 1M </pre>
返回值	成功返回 0，失败返回非 0
指令示例	<pre> iwpriv wlan0 common tx_rate,37 恢复自动速率命令 iwpriv wlan0 common tx_rate,255 </pre>

73 Echo 配置参数命令

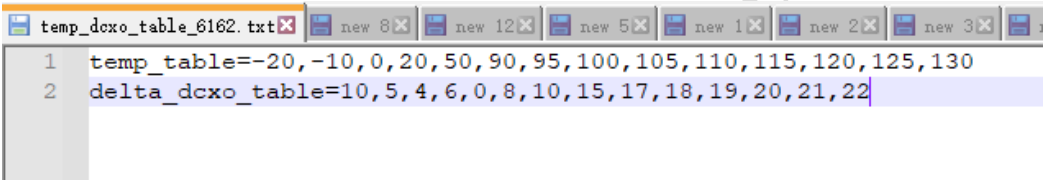
指令	echo 1 > /sys/module/atbm_mododul_namexxxx/parameters/xxx_xxx
参数	1. 开关功能: <pre>echo 1 打开</pre> <pre>echo 0 关闭</pre> 2. WIFI 驱动模块名字 atbm_mododul_namexxxx: 加载的 WIFI 驱动模块名字，输入前两个字母，按 tab 键会自动补齐 3. 配置参数 xxx_xxx: 具体的配置参数如下: beamforming_on: 开关 beamforming 功能（注意要在加载驱动之后连接 AP 之前配置该参数） tx_rate_static: 打开固定速率统计功能，跑固定速率时，看固件发包成功率统计打开这个选项

	he_not_40M: 支持 20M/40M 带宽时, 固定配置 TX 按 20M 带宽发送, 如果设置为 0 表示由速率控制自动切换, 设置为 1 表示强制设置为 20M 带宽 he_ltf_gi: HE 模式 GI 选择参数, 可配置 1, 2, 3, 4 恢复自动配置配 255 参数如下: ATBM_HE_4X32=1, ATBM_HE_4X08=2, ATBM_HE_2X16=3, ATBM_HE_2X08=4 he_dcm: 配置 0 关 DCM, 配置 1 开 DCM, 恢复自动配 255 (注意 DCM 需要协商支持才能配置), 协商支持后会自动配置 DCM, 一般不需要手工配置开 DCM, DCM 可以手工关闭, 如果打开 DCM, 需要选择固定速率, 并且只支持 MCS0,1,3,4 和 MCSER0,1,2 he_ldpc: 缺省值 1, 系统支持 HE 模式 LDPC 时自动选择 LDPC, 配置 0 关闭 LDPC。 如果系统 HE 模式不支持 LDPC, 该参数无效。 ht_ldpc: 缺省值 2 自动选择, 参数: 0 bcc; 1 ldpc; 2 auto 如果系统支持 HT 模式 LDPC 时配置 1 开 LDPC, 配置 0 关闭 LDPC。 如果系统 HT 模式不支持 LDPC, 该参数需要配置成 2 或 0。
功能描述	通过修改配置参数, 配置 WIFI 工作模式及速率统计等
返回值	成功返回 0, 失败返回非 0
指令示例	1、配置 HE 协商参数带 beamforming 信息, 打开 beamforming 功能 <pre>echo 1 > /sys/module/atbm_mododul_namexxxx/parameters/beamforming_on</pre> 2、打开固定速率统计功能 <pre>echo 1 > /sys/module/atbm_mododul_namexxxx/parameters/tx_rate_static</pre> 3、固定配置 TX 按 20M 带宽发送 <pre>echo 1 > /sys/module/atbm_mododul_namexxxx /parameters/he_not_40M</pre> 4、固定配置 HE GI 参数, 配置 2XP16 <pre>echo 3 > /sys/module/atbm_mododul_namexxxx /parameters/he_ltf_gi</pre>

74 设置频偏跟踪使能

指令	<code>iwpriv wlan0 fwcmd cfo,<enable></code>
参数	enable: 0 频偏不随 ap 跟踪功能; 1 设置频偏随 ap 跟踪功能; 2 设置频偏随 ap 跟踪功能; 3 关闭频偏跟踪功能, 开启温度动态调整 dcxo; (仅独立晶体有效)
功能描述	1 表示同时打开 AP 和 STA 端 cfo 校准 2 表示只打开 STA 端 cfo 校准 3 表示关闭频偏跟踪功能, 开启温度动态调整 dcxo; (仅独立晶体有效)
返回值	<code>cfo%d, dcxo%d</code> : 第一个参数打印输入参数 enable, 第二个参数打印当前 dcxo 值
指令示例	打开频偏随 ap 跟踪功能: <code>iwpriv wlan0 fwcmd cfo,0</code> 关闭频偏随 ap 跟踪功能: <code>iwpriv wlan0 fwcmd cfo,1</code>

75 根据温度调节频偏参数修改

指令	<code>iwpriv wlan0 common set_temp_dcxo_table</code>
参数	无
功能描述	根据实际情况修改温度门限和对应的 dcxo 补偿值保存到配置文件，然后将配置文件放到指定路径，然后使用 <code>set_temp_dcxo_table</code> 命令使新修改的配置生效。配置文件路径可以在 <code>hal_apollo/Makefile</code> 中配置，默认加载 <code>/tmp/</code> 下的 <code>temp_dcxo_table.txt</code> 文件： <pre>-DCOMPAT_STATIC ATBM_BUILD_TIME = \$(shell date "+%Y-%m-%d %H:%M:%S") ccflags-y += -DCONFIG_ATBM_BUILD_TIME=\"\$(ATBM_BUILD_TIME)\" ##指定配置文件路径，当配置文件存在时使用配置文件中的dcxo值不使用efuse中的dcxo，配置文件中的delta_gain为修正值，如果文件不存在则直接使用efuse中的数据 ATBM_CONFIG_FILE="/tmp/atbm_txpwr_dcxo_cfg.txt" ccflags-y += -DCONFIG_TXPOWER_DCXO_VALUE=\"\$(ATBM_CONFIG_FILE)\" # use the deltagain value in file :\"\$(ATBM_CONFIG_FILE)\",when file exist ATBM_RATE_POWER_CONFIG_FILE="/tmp/set_rate_power.txt" ccflags-y += -DCONFIG_RATE_TXPOWER=\"\$(ATBM_RATE_POWER_CONFIG_FILE)\" # use the temp table and dcxo table in file :\"\$(ATBM_TEMP_DCXO_TABLE_CONFIG_FILE)\",when file exist ATBM_TEMP_DCXO_TABLE_CONFIG_FILE="/tmp/temp_dcxo_table.txt" ccflags-y += -DCONFIG_TEMP_DCXO_TABLE=\"\$(ATBM_TEMP_DCXO_TABLE_CONFIG_FILE)\" ccflags-y += -DCONFIG_COUNTRY_VAL=\"\$(COUNTRY_VAL)\"</pre> 配置文件格式如下： 
返回值	无
指令示例	<code>iwpriv wlan0 common set_temp_dcxo_table</code>: 使配置文件生效 <code>iwpriv wlan0 fwcmd cfo,3</code>: 关闭频偏随 ap 跟踪功能同时开启温度动态调整 dcxo

76 读取寄存器

命令	<code>iwpriv wlan0 common rmem_bit,<reg_addr>,<end_bit>,<start_bit></code>
参数说明	<code>reg_addr</code> : 读取的寄存器地址（十六进制） <code>end_bit</code> : 寄存器高 bit（十进制） <code>start_bit</code>: 寄存器低 bit（十进制）
返回值	hmac 打印寄存器内容 <pre># # # iwpriv wlan0 common rmem_bit,acc017c,31,0 [38500.238261] [atbm_log]:cmd:rmem_bit,acc017c,31,0 , len=22 [38500.244514] [atbm_log]:comma=,acc017c,31,0, input_cmd_len = 8 [38500.250952] [atbm_log]:0Acc017C[31:0] 00000030 wlan0 common:rmem_bit,acc017c,31,0 # #</pre>
例子	启动接口 <code>ifconfig wlan0 up</code> 读取寄存器 <code>iwpriv wlan0 common rmem_bit,acc017c,31,0</code> 驱动打印：

	<pre># # # iwpriv wlan0 common rmem_bit,acc017c,31,0 [38500.238261] [atbm_log]:cmd:rmem_bit,acc017c,31,0 , len=22 [38500.244514] [atbm_log]:comma=,acc017c,31,0, input_cmd_len = 8 [38500.250952] [atbm_log]:0Acc017C[31:0] 00000030 wlan0 common:rmem_bit,acc017c,31,0 # #</pre>
--	--

77 写寄存器

命令	iwpriv wlan0 common wmem_bit,<reg_addr>,<end_bit>,<start_bit>,<reg_value>
参数说明	reg_addr : 读取的寄存器地址（十六进制） end_bit : 寄存器高 bit（十进制） start_bit: 寄存器低 bit（十进制） reg_value: 写入寄存器的值（十六进制）
返回值	无
例子	<pre>启动接口 ifconfig wlan0 up 读寄存器 iwpriv wlan0 common rmem_bit,16101410,7,0 写入寄存器 iwpriv wlan0 common wmem_bit,16101410,7,0,3b 再次读取确认 iwpriv wlan0 common rmem_bit,16101410,7,0 # # # iwpriv wlan0 common rmem_bit,16101410,7,0 [38793.881525] [atbm_log]:cmd:rmem_bit,16101410,7,0 , len=22 [38793.888008] [atbm_log]:comma=,16101410,7,0, input_cmd_len = 8 [38793.894506] [atbm_log]:16101410[7:0] 0000002D wlan0 common:rmem_bit,16101410,7,0 # # # iwpriv wlan0 common wmem_bit,16101410,7,0,3b [38841.569124] [atbm_log]:cmd:wmem_bit,16101410,7,0,3b , len=25 [38841.575804] [atbm_log]:comma=,16101410,7,0,3b, input_cmd_len = 8 [38841.582697] [atbm_log]:16101410[7:0] <= 0000003B wlan0 common:wmem_bit,16101410,7,0,3b # # # iwpriv wlan0 common rmem_bit,16101410,7,0 [38843.455594] [atbm_log]:cmd:rmem_bit,16101410,7,0 , len=22 [38843.462039] [atbm_log]:comma=,16101410,7,0, input_cmd_len = 8 [38843.468431] [atbm_log]:16101410[7:0] 0000003B wlan0 common:rmem_bit,16101410,7,0 # #</pre>

78 设计整机校准标识

指令	<code>iwpriv wlan0 common set_cali_flag,<flag></code>
参数	<code>flag</code> : 0: 关闭整机校准; 1: 开启整机校准 默认设置成 0
功能描述	cob 开启整机校准功率和频偏, 配置校准标识后会把 <code>deltagain</code> 和 TPC 寄存器复位
返回值	无
指令示例	<code>iwpriv wlan0 fwcmd set_cali_flag,1</code>

79 整机校准设置 `deltagain`

指令	<code>iwpriv wlan0 common set_deltagain,<deltagain></code>
参数	<code>deltagain</code> : 功率校准和复测时配置的数字增益补偿值, 有效范围 【-15, 15】 。 实际补偿的功率值是码字除以 4, 比如 <code>deltagain=4</code> , 则数字增益再增加 1dB。 默认设置成 efuse 中的 <code>deltagain</code> 值
功能描述	功率校准和复测时配置的数字增益补偿值。 注意: 必须要在发包后再配置, 仅仅在整机校准标识配置后生效。
返回值	无
指令示例	<code>iwpriv wlan0 common set_deltagain,4</code>

80 整机校准设置温度补偿

指令	<code>iwpriv wlan0 common set_tpc,<tjroom></code>
参数	<code>tjroom</code> : 整机校准时获取的结温 (功率校准时校准最后一个信道获取的温度) 单位: 摄氏度, 默认设置成 0
功能描述	功率复测时配置的温度补偿值。 注意: 必须要在发包后再配置, 仅仅在整机校准标识配置后生效。
返回值	无
指令示例	<code>iwpriv wlan0 common set_tpc,50</code>

81 读取当前 `dcxo`

指令	<code>iwpriv wlan0 common get_dcxo</code>
参数	无
功能描述	获取当前使用的 <code>dcxo</code> 值
返回值	无
指令示例	<code>iwpriv wlan0 common get_dcxo</code>

82 设置 dcxo

指令	<code>iwpriv wlan0 common set_dcxo,<dcxo></code>
参数	dcxo: 频偏调节因子, 有效范围【0, 127】 默认设置成 efuse 中的 dcxo 值
功能描述	设置当前使用的 dcxo 值
返回值	无
指令示例	<code>iwpriv wlan0 common set_dcxo,50</code>



CONTACT INFORMATION

AltoBeam (China) Inc.

Address: B808, Tsinghua Tongfang Hi-Tech Plaza, Haidian, Beijing, China 100083

Tel: (8610) 6270 1811

Fax: (8610) 6270 1830

Website: www.altobeam.com

Email: support@altobeam.com

DISCLAIMER

Information in this document is provided in connection with AltoBeam products. No license, express or implied, by estoppels or otherwise, to any intellectual property rights is granted by this document. Except as provided in AltoBeam's terms and conditions of sale for such products, AltoBeam assumes no liability whatsoever, and AltoBeam disclaims any express or implied warranty, relating to sale and/or use of AltoBeam products including liability or warranties relating to fitness for a particular purpose, merchantability, or infringement of any patent, copyright or other intellectual property right.

AltoBeam may make changes to specifications and product descriptions at any time, without notice. Designers must not rely on the absence or characteristics of any features or instructions marked "reserved" or "undefined." AltoBeam reserves these for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them. Unauthorized use of information contained herein, disclosure or distribution to any third party without written permission of AltoBeam is prohibited.

AltoBeam™ is the trademark of AltoBeam. All other trademarks and product names are properties of their respective owners.

Copyright © 2007~2025 AltoBeam, all rights reserved